树上问题

qwaszx

2026 年 9 月 18 日

目录

1 基本树上问题

树是最简单的图。序列可以视为一种特殊的树——链。相比序列，树失去了全序性质，分治结构也变得更加复杂。因此，树上问题往往要比序列问题复杂得多。

1.1 树的基本性质

无根树是一个无向图，具有以下几个基本性质：

- 无环，连通
- 点数减边数等于 1
- 任意两点之间有且仅有一条路径

如果去除连通性要求，那么就得到森林。森林具有以下几个基本性质：

- 无向无环图
- 连通块数等于点数减边数
- 任意两点之间至多有一条路径

第二条性质常常用来计算森林的连通块数，或者判断某个子图是否是树。

例 1.1. ABC390F. 给一个序列 a，定义区间 $[l,r]$ 的权值为 $a[l,r]$ 这些值形成的值域连续段个数。求所有区间的权值和。

练习 1.2. 给一棵树和一个序列 a，定义区间 $[l,r]$ 的权值为 $a[l,r]$ 在树上形成的导出子图连通块个数，求所有区间的权值和。提示

练习 1.3. 给一棵树和一个序列 a，定义区间 $[l,r]$ 的权值为 $a[l,r]$ 在树上形成的导出子图连通块个数。多次询问某个区间的权值和。

练习 1.4. [SDCPC 2019. Connected Intervals] 给一棵树和一个序列 a，定义区间 $[l,r]$ 的权值为 $a[l,r]$ 在树上形成的导出子图连通块个数。求有多少个区间的权值为 1。

如果指定某一个点作为根，则得到有根树。有根树导出了一个偏序关系：父节点大于子节点。在这个偏序关系中，两个点可比（即 $x<y$ 或者 $x>y$）当且仅当它们之间有祖先后代关系。任意点集的公共祖先集合都是一条从根出发的链，其中深度最大的点称为它们的最近公共祖先（Lowest Common Ancestor，LCA）。有根树的深度定义为从根到该点的路径长度（有时也定义为路径上的点权和）。

1.1.1 树的子结构

树的常见子结构有以下几种：子树、路径/链、邻域。它们都是特殊的连通子图。

定义 1.5. 在有根树中，**子树**是某个结点及其所有后代导出的子图。无根树的子树则是一个有歧义的概念，既可以指一个连通块，也可以指不包含某个点的极大连通块。因此，我们不对无根树使用这个术语。
路径/链指的是某两个点之间的唯一路径。在有根树中，链可以分为直上直下的链和非直上直下的链两类。前者的性质通常与序列相似。任何一条链都可以由不超过两条直上直下的链拼起来得到。
点 u 的 k-**邻域**定义为距离 u 不超过 k 的点集。

给定点集 S，在树上包含它们的最小连通块是唯一的，这个连通块通常被称为**虚树**。

1.2　树的序列化

可以通过某种方式遍历树来得到一个序列。根据遍历方式的不同，得到的序列会有不同的性质。

1.2.1　DFS 序

DFS 序是最常用的遍历顺序，它的性质是子树内的点在 DFS 序中是一段连续的区间。因此，可以通过这种方式将子树问题转为区间问题。不过，子树问题通常要比区间问题更容易。结点在 DFS 序中的位置通常用时间戳 `dfn[u]` 表示。

1.2.2　BFS 序

树的 BFS 序是用得比较少的遍历顺序。它的性质是一个点的所有孩子在 BFS 序中连续。对 BFS 序的一种更灵活的理解方式是把树按深度分层。

例 1.6. 邻域修改查询. 给一棵树，支持把一个点的 k-邻域内的点权加，查询一个点的 k-邻域内的点权和。$k = O(1)$。

1.2.3　欧拉序

欧拉序是树按照一个“欧拉回路”访问点得到的顺序。在有根树中，可以这样得到一个欧拉序：

```
1 void dfs(int u) {
2     euler.push_back(u);
3     for (int v : son[u]) {
4         dfs(v);
5         euler.push_back(u);
6     }
7 }
```

性质 1.7. 在欧拉序中，取 u 和 v 的任意一对出现位置，则这两个位置之间的区间会包含 u, v 链上的所有点的至少一次出现，且不包含 $\mathrm{lca}(u, v)$ 子树之外的点。因此，这段区间中深度最小的点全都是 $\mathrm{lca}(u, v)$。

1.2.4　括号序

括号序是在 DFS 进入和离开结点 u 时分别写下 u^+ 和 u^- 得到的序列。设 $l = \mathrm{lca}(u, v)$。括号序的性质是：假设 u 在 v 前被访问到，则括号序列在 u^+（不含）和 v^+（含）之间的部分抵消掉成对的 w^+ 和 w^- 后，剩下的是 $[u, l)$ 这条链上的 w^- 和 $(l, v]$ 这条链上的 w^+。通过这种方式可以把一些链上问题变成序列上的问题。

例 1.8. 括号序树链莫队. 给一棵带颜色的树，多次询问一条链上有多少种出现次数为奇数的颜色。

评注. 完全的树链莫队需要树分块。

1.3　LCA 的计算

有根树上的各种计算往往都需要 LCA。这里给出 LCA 的若干计算方式。

1.3.1　向上跳

首先考虑一个暴力：将深度更大的那个点向上跳到与另一个点深度相同，然后在两个点相遇之前不断地一起往上跳。我们可以通过一些手段优化这个暴力：

- 倍增：预处理 $f_{i,j}$ 表示点 i 向上跳 2^j 步跳到的点即可。预处理 $O(n \log n)$，单次询问 $O(\log n)$，空间 $O(n \log n)$。
- 链剖分：在两个点跳到同一条链之前，每次让链顶深度更大的点向上跳到链顶的父亲；跳到同一条链之后返回深度小的点。通常使用重链剖分，此时预处理 $O(n)$，单次询问 $O(\log n)$，且常数较小。

评注. 这两种方法较难做到 $O(1)$ 询问。倍增法的空间可以通过斜二倍增优化到 $O(n)$。

1.3.2　RMQ

利用树的序列化可以将 LCA 问题变成 RMQ 问题。

- 欧拉序：直接使用欧拉序找到两点在欧拉序中第一次出现的区间，然后在这个区间内查询深度最小的点。
- DFS 序：设 $\mathrm{dfn}[u] < \mathrm{dfn}[v]$，则 dfn 在 $(\mathrm{dfn}[u], \mathrm{dfn}[v]]$ 中的点都在 $\mathrm{lca}(u, v)$ 的子树（不含本身）内，且其中必然包含 $\mathrm{lca}(u, v)$ 的一个儿子。因此，取 $\mathrm{dfn}[fa_x]$ 最小的 $\mathrm{fa_x}$ 即可。使用这种方法容易做到 $O(1)$ 询问。

1.4　树的子结构的性质

1.4.1　子树

在已知 S 的导出子图连通的情况下，可以进一步检查

$$\sum_{u\in S}\deg(u)=2|S|-1.$$

是否满足来判断 S 是否为一个非整棵树的子树。

删去一个点集 S 后，剩余图仍然连通，当且仅当 S 的每个连通分量都至多通过一条边与 $V\setminus S$ 相连。等价地，若保留下来的点集 $C=V\setminus S$ 非空，则 C 有唯一一个深度最小的点 r_C，且 C 中任意点到 r_C 的路径都完全包含在 C 中；这时 S 可以看成 $V\setminus\mathrm{subtree}(r_C)$ 加上若干个从 C 上剪掉的完整子树。

练习 1.9. P8990.

子树之间要么包含，要么不交。反过来，具有这种性质的集合族也可以构建为一棵树。

1.4.2　路径/链

练习 1.10. 判断点是否在链上. 给定点 u,v,x，判断 x 是否在 u 到 v 的链上。

解答. 方法一：检查 $d(u,x)+d(x,v)=d(u,v)$。

方法二：将链分解为两条直上直下的链上分别判断。直上直下的链上的判定可以转为子树判定，这可以通过 DFS 序实现。

性质 1.11. 两条链的交要么是空集，要么仍然是一条链。具体来说，可以这样计算 (u_1,v_1) 和 (u_2,v_2) 的交：考虑 $\mathrm{lca}(u_1,u_2),\mathrm{lca}(u_1,v_2),\mathrm{lca}(v_1,u_2),\mathrm{lca}(v_1,v_2)$ 这四个点中按重数取深度最大的两个点 u,v（可能相同），那么链交要么是 (u,v)，要么是空集，可以通过判断 u 是否同时在 (u_1,v_1) 和 (u_2,v_2) 上来区分。

1.4.3　虚树

点集 S 的虚树有一个紧凑的刻画方式。

性质 1.12. 固定点集 S。树上的一个点 u 为关键点当且仅当 $u\in S$，或者 u 有至少两个儿子的子树与 S 有交。一个关键点的父关键点定义为其祖先中深度最大的关键点。则 S 的虚树是所有关键点和其父关键点之间的链的边不交并。把每条这样的链压成一条边，就得到通常意义下的虚树。

练习 1.13. 令 $\bar{S}=\{\mathrm{lca}(u,v):u,v\in S\}$，说明关键点集合即为 $\bar{S}$。

练习 1.14. 证明关键点集合的大小不超过 $2|S|-1$。

例 1.15. 树上 TSP. 给一棵树和一个点集 S，选择 S 的一个排列 $u_1, \ldots, u_m$，分别使得

- $d(u_1, u_m) + \sum_{i=2}^{m} d(u_{i-1}, u_i)$ 最小
- $\sum_{i=2}^{m} d(u_{i-1}, u_i)$ 最小

例 1.16. 链并. 给一棵树和树上的 m 条链，将这些链的并集表示为 $O(m)$ 条不交的直上直下的链的并。

1.5　树上差分与前缀和

对于可差分的问题，可以将链用两类性质较好的集合表示：从根出发的链，以及子树。这两者互为对偶。

一条链可以表示为两条直上直下的链的不交并（且上端点大致相同），而直上直下的链又可以表示成两条从根出发的链的差。通过这种方式，我们可以把任意一条链表示为至多四条从根出发的链的加减。

记 l_v 为从根到 v 的链，s_u 为 u 的子树，则 $u \in l_v$ 当且仅当 $v \in s_u$。通过这种方式，可以在这两种范围之间做对偶。例如，链加、查询单点可以对偶到单点加、查询子树和。

在 $\{l_u\}$ 结构上存在自然的前缀和与差分算子：$Sf(u) = Sf(\mathrm{fa}_u) + f(u)$，$Df(u) = f(u) - f(\mathrm{fa}(u))$，容易验证它们互逆。在 $\{s_u\}$ 结构上的前缀和与差分算子则更加复杂一些：$Sf(u) = f(u) + \sum_{v \in \mathrm{son}_u} Sf(v)$，$Df(u) = f(u) - \sum_{v \in \mathrm{son}_u} f(v)$。我们在组合数学课程中学习的前缀和与差分技巧也可以在这里使用。

练习 1.17. 应用一. 给定一个 LCA 黑盒。在 $O(1)$ 时间内查询一条链的点权和以及边权和，或者某个子树的点权和以及边权和。

练习 1.18. 据此得到一个 $O(1)$ 查询两点间距离的算法。

例 1.19. 应用二. 给定一个 LCA 黑盒。在 $O(n+q)$ 时间内支持：一条链上的点权/边权加，某个子树内的点权/边权加，在所有修改结束后输出所有点权和边权。

例 1.20. P4211. 给一棵树，多次询问

$$\sum_{i=l}^{r} f(\mathrm{lca}(i, x)).$$

（假设有一个支持链修改查询的黑盒）

评注. 此题还有线段树套虚树、点分树等做法。

2 重心和直径

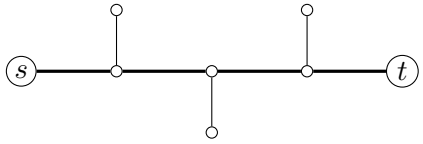
2.1 直径

树的直径定义为距离最远的两点之间的距离。这里距离定义为两个点之间唯一路径上的边权和/点权和/边权和与点权和的和。我们也把这样的两个点之间的路径称为直径。直径可能存在多条。

评注. 在有负权时，这样定义的距离的性质通常比较差。如无特殊说明，我们默认树无负权。

评注. 在树退化成一条链的情况下，求直径相当于求最大子段和。这可以说明一些树上问题的困难性，如链加、查询直径。

在确定直径后，可以将树画成以下的形式来帮助直观思考：



有以下性质成立：

性质 2.1. 假设树无负权。任取一条直径 (s,t)，则对于任何一个点 u，s,t 中至少有一个是距离 u 最远的点之一。换句话说，距离一个点最远的点一定可以是直径的一个端点。

练习 2.2.

- 说明在有负权时不成立。
- 在边权和点权均非负，且距离 $d(u,v)$ 定义为路径上边权和加上 u,v 两点的点权和时，证明该性质仍然成立。

练习 2.3. 据此得到无负权树的直径的算法。

练习 2.4. 在有负权时，使用 DP 来计算直径。

下面的性质给出了连接两棵树后直径的变化。

性质 2.5. 假设树 T_1 的一条直径是 (s_1,t_1)，树 T_2 的一条直径是 (s_2,t_2)，并且边无负权。如果在两棵树中间加一条边得到一棵新树，那么新树中存在一条两个端点都在集合 $\{s_1,t_1,s_2,t_2\}$ 内的直径。

练习 2.6. 说明如何加边才能使得新树的直径最大/最小。

练习 2.7. 支持加边（保证任意时刻是森林）、查询某个连通块的直径。

练习 2.8. P3596. 给一棵树，对每条边，计算删除这条边然后再加一条新边后形成的树的直径可能的最大值和最小值。

对于边权为正的树，还有以下结论：

性质 2.9. 所有直径的中点（允许在边上）都相同。这个点称为树的中心（center），它最小化到树上所有点的最大距离。

评注. 对于有零边的情况，可以把零边缩起来。

练习 2.10. P1099/P2491. 给定一棵非负权的树 T 和一个常数 s，求一对点 (u,v) 使得 $d(u,v)\le s$ 且 $\max_x \min_{y\in P_{uv}} d(x,y)$ 最小。也就是找到一条长度不超过 s 的路径，使得到这条路径的最远距离最小。需要 $O(n)$。

练习 2.11. P3304. 询问有多少条边满足所有的直径都经过它。

我们也可以定义一个点集内的直径。对于点集 S，定义 S 内的直径为 S 中距离最远的两点之间的距离。此时，有以下性质成立。

性质 2.12. 假设树无负权。任取点集 S 内的一条直径 (s,t)，则对于树上的任何一个点 u，s,t 中至少有一个是 S 内距离 u 最远的点之一。

练习 2.13. 证明以下结论：任取点集 S_1 内的一条直径 (s_1,t_1) 和点集 S_2 内的一条直径 (s_2,t_2)，则 $S_1\cup S_2$ 内存在一条两个端点都在 $\{s_1,t_1,s_2,t_2\}$ 内的直径。因此，点集内的直径是一种可以快速合并的分治信息。

练习 2.14. 多次询问 $S=[l,r]$ 内的直径。

练习 2.15. 给一棵树，维护一个点集 S，支持添加/删除点到 S 中，查询 S 内的直径。

评注. 有负权时可以通过动态 *DP* 之类的方法做到 $O(\log n)$，参见 *P4115*。

例 2.16. CF1192B. 修改边权，查询直径。保证边权非负。

评注. 如果没有边权非负的保证，那么可以用静态 *Top Tree* 等方法做到 $O(\log n)$。

2.2　重心

假设树 T 带有非负的点权，且总点权大于零。定义一个点集 S 的重量 $w(S)$ 为该点集内的点权和。树的重心（median/centroid）有以下两个等价定义：

- 以重心 u 为根时，$\max_{v\in \mathrm{son}_u} w(\mathrm{subtree}(v))$（最大的子树重量）最小。
- 以重心 u 为根时，任何 $v\neq u$ 都满足 $w(\mathrm{subtree}(v))\le w(T)/2$（每个子树的重量都不超过总重量的一半）。

评注. 注意重心未必在直径上。

直观上，重心可以这样取得：假设当前在点 u，考虑删掉 u 形成的所有连通块，如果存在重量 $> w(T)/2$ 的那么就往这个连通块移动，否则 u 本身就是重心。利用这个直观，我们可以得到以下性质：

性质 2.17.

- 对一棵以 r 为根的树，若 r 不是重心，则存在唯一一个重量 $> w(T)/2$ 的儿子子树，并且所有重心都在这棵子树中。
- 假设 u 是树的重心，并把树以 u 为根。那么树除了 u 之外的所有重心，恰好是满足 $w(\text{subtree}(v)) = w(T)/2$ 的点 v。在点权均为正的情况下，这样的点 v 至多只有一个，故此时重心至多有两个且二者相邻。

对于一棵以 r 为根的有根树，在切换到以 u 为根时，除了原本的子树外，还会产生一个“向上”的子树，这个子树的重量为 $w(T) - w(\text{subtree}(u))$。可以据此判定一个点是否为树的重心。

练习 2.18. P5666. 给一棵树，计算删除每条边后，形成的两个连通块中的所有重心的编号之和。

下面的性质将重心与距离联系在一起。

性质 2.19. 假设树的边权非负，距离定义为两点之间的路径的边权和。树的重心可以最小化所有点到它的距离之和，即 $\sum_v w_v \cdot d(v,u)$。

也可以据此定义点集内的重心：

定义 2.20. 对于点集 S，定义 S 内的重心为距离 S 内所有点的距离之和最小的 S 内的点，即 $\arg\min_{u\in S} \sum_{v\in S} w_v \cdot d(u,v)$。这里距离定义为两点之间路径上的边数（事实上任何非负边权导出的距离都可行）。

下面的性质说明了连接两棵树后重心的变化情况。

性质 2.21. 假设树 T_1, T_2 的点权均为正，且 u_1, u_2 分别是它们的一个重心。如果在两棵树中间加一条边得到一棵新树，那么新树的所有重心都在 u_1 和 u_2 之间的链上。

例 2.22. P4299. 加边（保证任意时刻是森林），查询连通块重心。

下面的性质给出了重心与 DFS 序的联系。

性质 2.23. DFS 序的带权中点定义为满足 $\sum_{i<k} w_{p_i} \le \frac{W}{2}$ 且 $\sum_{i>k}^{n} w_{p_i} \le \frac{W}{2}$ 的 k。这样的 p_k 在深度最小的那个重心的子树内。

练习 2.24. 子树点权加/赋值，查询重心。

评注. 后面我们学习链剖分之后还能支持链修改。

练习 2.25. 查询 $S=[l,r]$ 内的重心。

解答. 通过可持久化线段树/整体二分之类的方法求出 S 在 DFS 序中的带权中点，然后从那个点向上跳到最后一个子树内有至多 $w(S)/2$ 重量的点即可。

2.3　静态树上问题

2.4　链剖分

链剖分把树的**点集**划分为若干条直上直下的链，两个端点在同一条链中的边称为链内边/重边/实边，不在同一条链中的边称为链间边/轻边/虚边。也可以用另一种方式理解这个链划分：每个结点选择至多一个儿子作为特殊儿子，对应的边为链内边，其它边为链间边。根据选择儿子的方式不同，可以得到以下几种链剖分：

- 重链剖分：每个结点选择子树大小最大的儿子作为特殊儿子，称为重儿子。此时，链内边称为重边，其它边称为轻边，划分出的链称为重链。
- 长链剖分：每个结点选择子树深度最大（指从根到最深叶子的长度）的儿子作为特殊儿子，称为长儿子/重儿子。此时，链内边称为长边/重边，其它边称为短边/轻边，划分出的链称为长链。
- LCT 中使用的动态链剖分：每个结点选择任意一个儿子作为特殊儿子，称为实儿子。此时，链内边称为实边，其它边称为虚边，划分出的链称为实链。

在下文中，我们称链剖分中的一条链为**实链**，而链则用于指代普通的树上路径。son_u 表示结点 u 的特殊儿子，child_u 表示结点 u 的所有儿子组成的集合。一条实链中最浅的结点称为这条实链的链顶，u 所在实链的链顶记作 $\mathrm{top}[u]$。

链剖分的基本性质是每条链间边的较深结点都是一条实链的链顶；反过来，一条实链的链顶要么是根，要么是某条链间边的较深结点。不过，需要注意链间边的较浅结点未必是一条实链的链底。根据这个性质，有以下结果成立：

$$\sum_u \sum_{v\in \mathrm{child}_u\setminus\{\mathrm{son}_u\}} v \text{ 到所在实链底的距离} \le \sum_{\text{链}C} |C| = n.$$

这是分析复杂度时常用的一个结论。

练习 2.26. 令 l_u 为 u 到其子树内最深叶子的长度，即长链剖分的划分依据。在长链剖分中，证明

$$\sum_u \sum_{v\in \mathrm{child}_u\setminus\{\mathrm{son}_u\}} l_v \le n.$$

这是长链剖分优化 DP 的一个依据。

链剖分的一个简单应用是寻找同一条实链上某个深度的点。

例 2.27. 给定一个链剖分，$O(1)$ 回答从 u 出发沿着其所在链向上/下走 k 步到达的点，或者报告离开了所在链。

练习 2.28. k 级祖先. 给一棵有根树，多次询问一个点的 k 级祖先。离线 $O(n)-O(1)$，在线 $O(n\log n)-O(1)$。

练习 2.29. 给一棵有根树，儿子有序排列。多次询问从某个点出发，每次都走最小的儿子，走 k 步后到达的点。保证询问点 u 子树内任何叶子到 u 的距离都至少为 k

任何一条直上直下的链都可以自下而上地分解为如下结构：（实链前缀、链间边）重复若干次，再加上一个实链的一段。如果是到根的链，则最后一段也是一个实链前缀。我们可以对每条实链维护区间数据结构，从而支持一些链信息查询。我们也可以利用实链快速上跳来计算 LCA。

算法 2.30. 链剖分计算 LCA. 在 u,v 处于同一条实链前，反复执行以下过程：每次从 u,v 中选择链顶深度较大的一个，不妨设为 u，将 u 跳到 $fa[\mathrm{top}[u]]$。对于同一条实链上的 u,v，LCA 即为深度较小的那个。复杂度为 u,v 路径上链间边的个数。

练习 2.31. 对于线段树可以直接 $O(\log n)$ 维护的静态区间信息查询，设计一个 $O(u,v$ 路径上的链间边个数 $+\log n)$ 的算法来支持树路径上的相同信息查询。

练习 2.32. 对于线段树可以直接 $O(\log n)$ 维护的带修区间信息查询，设计一个 $O(u,v$ 路径上的链间边个数 $\log n)$ 的算法来支持树路径上的相同修改查询。

评注. 链剖分 + 线段树并不是一个完全平衡的结构，这导致复杂度有一个额外的 log 因子。可以使用全局平衡二叉树来做到 $O(\log n)$ 修改查询。

显然，链剖分的复杂度分析依赖于链间边的个数。不同的链剖分方式会导致不同的复杂度。

引理 2.33. 在重链剖分中，任意根到点的路径上经过的轻边数不超过 $\log_2 n$。

证明. 从上往下每经过一条轻边，子树大小至少翻倍。 □

练习 2.34. 证明长链剖分中，任意根到点的路径上经过的短边数为 $O(\sqrt{n})$。

可以配合 DFS 序来同时支持维护子树信息。在 DFS 时，我们优先递归 son_u，这样就能保证一条重链在 DFS 序中是连续的。按照这种 DFS 序构建的线段树能够同时支持重链和子树的修改查询。我们也可以设计其他的结点排序方式来支持更复杂的询问。

例 2.35. 毛毛虫剖分. 对于路径 P_{uv}，$\{x : d(x,P_{uv}) \le 1\}$ 称为 u,v 的毛毛虫。构造一个结点排序方式，使得子树可以划分为 $O(1)$ 个连续区间，且一个毛毛虫可以划分为 $O(\log n)$ 个连续区间。

解答. 重链剖分，然后按照如下顺序构造顺序：写下所有儿子，递归重儿子，然后递归所有轻儿子。这样，对于每条重链 C，设其自上而下为 $u_1, \cdots, u_k$，则递归到 u_1 时会连续写下 $\text{child}_{u_1}, \text{child}_{u_2}, \cdots$。也就是说一条重链对应的毛毛虫中，除了链顶之外都是连续的。同理，对于子树，除了子树根以外都是连续的。

练习 2.36. k-毛毛虫剖分. 对于路径 P_{uv}，$\{x : d(x, P_{uv}) \le k\}$ 称为 u, v 的 k-毛毛虫。构造一个结点排序方式，使得子树可以划分为 $O(k)$ 个连续区间，且一个 k-毛毛虫可以划分为 $O(k \log n)$ 个连续区间。

2.5 Top Tree

3　树分治